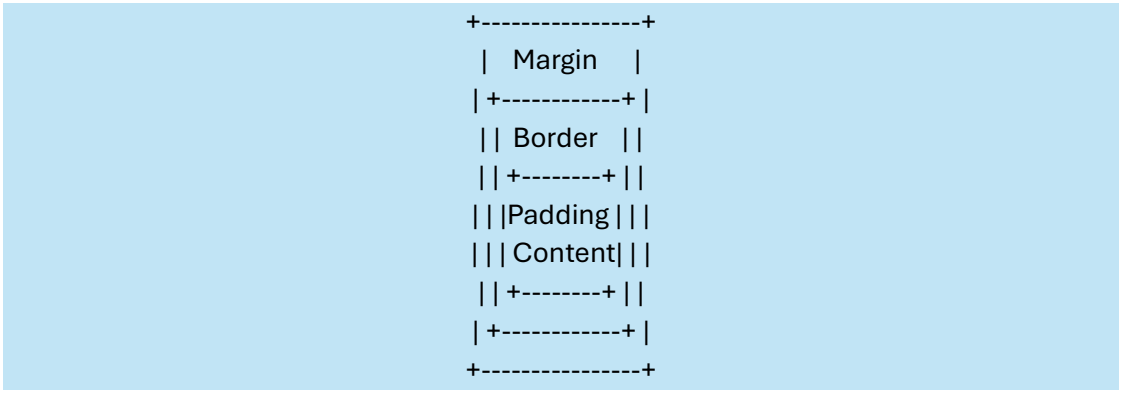
Every HTML element is treated as a box containing:



Box Model Components:

- Content
- Padding
- Border
- Margin

Types of CSS Box Layouts

1. Block Layout
2. Flexbox Layout
3. Grid Layout

Part 1: CSS Box Model

Step 1: Create a simple box

index.html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="box">Hello CSS Box</div>

</body>
</html>
```

style.css

```
.box{
  width:200px;
  height:100px;
  background:lightblue;
}
```

Understand width , height from above

Step 2: Add padding

```
.box{
  width:200px;
  height:100px;
  background:lightblue;
  padding:20px;
}
```

We can observe:

- Text moves away from the edges.

Description:

- Padding is inside the box.

For understanding:

```
+----------------------+

| Padding || +--------------+ || | Content | || +--------------+ |

+----------------------+
```

Step 3: Add border

```
.box{
  width:200px;
  height:100px;
  background:lightblue;
  padding:20px;
  border:5px solid black;
}
```

Description:

- Border surrounds the padding.

Step 4: Add margin

```
.box{
  width:200px;
  height:100px;
  background:lightblue;
  padding:20px;
  border:5px solid black;
  margin:30px;
}
```

If a duplicate box is created:

```
<div class="box">Box 1</div>
<div class="box">Box 2</div>
```

Observation:

- Margin creates space between elements.

Quick Formula

Total Width=Width+Padding+Border

Example:

- width = 200
- padding = 20 left + 20 right
- border = 5 left + 5 right

Total = 250 px

Part 2: Block Layout

Every div is a block element.

```
<div class="block">Header</div>
<div class="block">Content</div>
<div class="block">Footer</div>
```

```
.block{
  width:300px;
  height:80px;
  background:lightgreen;
  border:2px solid black;
  margin:10px;
  padding:10px;
}
```

Question:

- Why are boxes appearing one below another?

Answer: Because div uses display:block by default.

Description:

Block layout = vertical layout

```
[Header]
[Content]
[Footer]
```

Part 3: Flexbox Layout

Block layout is good for sections.

But what if we want products in a row?

Step 1: Create a container

index.html

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
</div>
```

style.css

```
.container{
  display:flex;
}

.item{
  width:100px;
  height:100px;
  background:orange;
  border:2px solid black;
}
```

What changed?

Answer:

- Boxes are now in a row.

Description:

display:flex converts the container into a flex container.

Step 2: Add spacing

```
.container{
  display:flex;
  gap:20px;
}
```

Description:

- gap adds space between items.

Step 3: Center items

```
.container{
  display:flex;
  justify-content:center;
  gap:20px;
}
```

description:

- justify-content controls horizontal alignment.

Step 4: Column layout

```
.container{
  display:flex;
  flex-direction:column;
  gap:10px;
}
```

Description:

- row = horizontal
- column = vertical

Step 5: Wrap

```
.container{
  display:flex;
  flex-wrap:wrap;
  gap:10px;
}

.item{
  width:120px;
  height:80px;
  background:orange;
  border:2px solid black;
}
```

Add more boxes.

We will see:

```
[1] [2] [3]
[4] [5] [6]
```

Description:

Flexbox is ideal for:

- Navigation bars
- Product rows
- Cards
- Toolbars

Which is easier for a row of products?

Answer: Flexbox

Part 4: Grid Layout

Flexbox is 1-dimensional.

Grid is 2-dimensional (rows and columns).

Step 1: Basic Grid

index.html

```
<div class="grid">
  <div class="cell">1</div>
  <div class="cell">2</div>
  <div class="cell">3</div>
  <div class="cell">4</div>
  <div class="cell">5</div>
  <div class="cell">6</div>
</div>
```

style.css

```
.grid{
  display:grid;
  grid-template-columns:200px 200px 200px;
  gap:10px;
}

.cell{
  height:100px;
  background:skyblue;
  border:2px solid black;
}
```

Description:

- 3 columns
- Automatic rows

Result:

```
[1] [2] [3]
[4] [5] [6]
```

Step 2: Equal columns

```
.grid{
  display:grid;
  grid-template-columns:1fr 1fr 1fr;
  gap:10px;
}
```

Description:

1fr means equal fraction.

Step 3: Responsive Grid

```
.grid{
  display:grid;
  grid-template-columns:repeat(auto-fit,minmax(150px,1fr));
  gap:10px;
}
```

Now resize the browser.

We will observe:

- 4 columns
- then 3
- then 2
- then 1

This is a real responsive gallery.

Step 4: Simple Webpage Layout

index.html

```
<div class="layout">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="main">Main Content</div>
  <div class="footer">Footer</div>
</div>
```

style.css

```
.layout{
  display:grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns:200px 1fr;
  grid-template-rows:60px 1fr 60px;
  height:100vh;
}

.header{
  grid-area:header;
  background:#333;
  color:white;
}

.sidebar{
  grid-area:sidebar;
  background:#ddd;
}

.main{
  grid-area:main;
  background:#fafafa;
}

.footer{
  grid-area:footer;
  background:#333;
  color:white;
}
```

Visual representation:

```
+---------+----------------------+
| Header (spans both columns) |
+---------+----------------------+
| Sidebar | Main Content       |
| 200px     | Remaining Width |
+---------+----------------------+
| Footer (spans both columns) |
+---------+----------------------+
```

This resembles real websites.

Task:

Photo Gallery.

Requirements:

- 6 boxes
- Equal spacing
- Responsive

Expected solution:

```
.gallery{
   display:grid;
   grid-template-columns:repeat(auto-fit,minmax(150px,1fr));
   gap:15px;
}

.photo{
   height:120px;
   background:lightpink;
   border:2px solid black;
}
```